

Grama-Angana

Community Space Manager – Digital Hub for Village Halls & Samudaya Bhavanas

Project Title

Android App Development using GenAI – Grama-Angana (Infrastructure) – Community Hall Booking & Maintenance Crowdfunding App

Project Number

49

Platform

Android (Kotlin + XML Layouts)

Domain

Infrastructure / Community Management / E-Governance

Category

Android Application with Firebase Realtime Database, Room Database & CalendarView

Report Type

Internship / Academic Project Report

PREPARED BY:

ARSHID AHMAD MALIK

USN: 1MJ23EC401

1MJ23EC401@MVJCE.EDU.IN

MVJ COLLEGE OF ENGINEERING

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

2025 – 2026

1. Introduction

Grama-Angana is an Android application developed to help Gram Panchayats, local youth groups, self-help organisations, and individual community members across India's villages and towns digitally manage the booking, scheduling, and maintenance of community halls, Anganas, and Samudaya Bhavanas. The application name combines Kannada words — Grama (village) and Angana (courtyard or community space) — reflecting its purpose as the trusted digital manager of every village's shared public infrastructure.

Developed using Kotlin with XML-based layouts, the app integrates Firebase Realtime Database for live booking request synchronisation across multiple users, Room Database for offline maintenance item storage, a custom CalendarView with colour-coded status markers for visual availability display, and Android's NotificationManager for booking status alerts. The application targets the vast network of government-built community halls across India that remain underutilised because residents have no simple way to check availability, request a slot, or report maintenance needs.

The application addresses four critical needs of village communities: providing a publicly viewable event calendar showing booked and free hall slots to eliminate scheduling conflicts; enabling residents to submit booking requests digitally to the Panchayat without visiting the office; maintaining a crowdfunded Maintenance Jar where community members can pledge support for small repairs; and displaying a real-time Event Board showing today's scheduled activities so residents know what is happening in their village.

Grama-Angana is designed with a dual-mode architecture: a publicly accessible Community View for residents that requires no login, and a PIN-secured Admin View for Panchayat officials to approve or reject booking requests and manage maintenance items. Booking data is synchronised in real time via Firebase, ensuring that all residents see the same up-to-date availability calendar. Maintenance item data is stored locally using Room Database for offline resilience. The app targets Gram Panchayat offices and community members across rural Karnataka and India who currently manage hall bookings through paper registers and informal phone calls.

2. Problem Statement

India has invested heavily in building community halls, Samudaya Bhavanas, and Anganas across its villages under schemes such as the Backward Regions Grant Fund and various state rural development programmes. Despite this investment, a large proportion of these halls remain empty or severely underutilised. Local youth, self-help groups, and cultural organisations struggle to use these spaces because of a broken booking and management process. Several systemic problems affect this segment:

Challenge	Description
No Availability Visibility	Residents have no way to know whether the hall is free on a given date without visiting the Panchayat office in person, often to be told it is already booked.
Double Booking Conflicts	Paper-based booking registers allow multiple parties to be assigned the same slot due to clerical errors, leading to conflict and social tension on event days.
Informal Key Access	The identity and contact details of the key holder are not publicly known, forcing residents to rely on word-of-mouth to find who can open the hall, often causing delays.
Neglected Maintenance	Broken fans, missing chairs, leaking roofs, and non-functional lights go unrepaired for months because there is no transparent mechanism for reporting needs or pooling community funds for small repairs.
Favouritism in Booking	Without a transparent, publicly visible booking system, hall access is often granted on the basis of personal connections rather than a fair first-come-first-served process, eroding community trust.
Building Decay	Under-use and deferred maintenance cause community halls to fall into disrepair, wasting public investment and depriving communities of a vital shared resource.

These interconnected challenges highlight the urgent need for a dedicated, transparent, and real-time community hall management application that Panchayat officials and residents can use with minimal

technical knowledge, on low-cost Android smartphones, to transform an idle public asset into a thriving hub of community activity.

3. Project Vision

Grama-Angana envisions a world where every village community hall — regardless of its size, location, or the technical capability of the local Panchayat — is transparently managed, actively used, and well maintained through the power of a simple, affordable Android application.

The project is built on four foundational pillars:

- **Transparency** – The booking calendar must be publicly visible to every community member at all times. No resident should lose access to the hall because they did not know it was available, and no official should be able to grant bookings without a visible, time-stamped record.
- **Accessibility** – The application must work on affordable Android devices in rural settings with intermittent connectivity. The public-facing Community View must be usable without login, and offline maintenance features must remain functional without internet access.
- **Community Ownership** – The hall belongs to the community. The Maintenance Jar crowdfunding feature gives every resident a direct stake in keeping the shared space functional, replacing passive neglect with active collective responsibility.
- **Simplicity** – Both the resident interface and the admin interface must be usable by individuals with minimal digital experience. The calendar is the central metaphor: intuitive, visual, and universally understood.

Grama-Angana is not merely a booking app; it is a civic empowerment tool designed to restore community trust in shared public infrastructure and turn an empty village hall into a living, breathing hub of social, cultural, and educational activity — one booking at a time.

4. Scope of the Application

4.1 Target Users

- Gram Panchayat secretaries and ward members who manage hall booking approvals and maintain the community space
- Local youth groups, sports clubs, cultural organisations, and self-help groups who want to book the hall for events, training, and practice sessions
- Individual community members who wish to book the hall for weddings, funerals, meetings, or private celebrations
- Community residents who want to contribute to the Maintenance Jar for small repairs such as new fans, chairs, or electrical fittings
- Village health workers and NGOs who use the hall for periodic health camps, awareness drives, and skill training programmes

4.2 Platform

- Android smartphones with minimum API Level 21 (Android 5.0 Lollipop and above)

- Optimised for low-end and mid-range devices with limited RAM and storage
- APK size kept minimal to support devices with limited internal storage and slow rural internet connections

4.3 Online and Offline Functionality

- Booking request submission and calendar synchronisation require a mobile data or Wi-Fi connection via Firebase Realtime Database
- Maintenance item list, pledge tracking, and Event Board data are stored locally using Room Database and remain accessible offline
- The public Community View (calendar and Event Board) loads the last cached Firebase data when offline, ensuring basic visibility even without connectivity

4.4 Key Operational Scenarios

- A resident opening the app and viewing the colour-coded calendar to check which dates are free for a wedding booking
- A youth group submitting a booking request form for a sports practice slot on a specific date and time
- A Panchayat admin logging in with a PIN, reviewing pending booking requests, and approving or rejecting them with a confirmation dialog
- A community member viewing the Maintenance Jar list, seeing that a new fan is needed for Rs. 500, and pledging a contribution
- A resident checking the Event Board to see that a health camp is scheduled at the hall today

5. Objectives

The project is aligned with the following measurable objectives:

- **Deliver a real-time, publicly visible booking calendar** – Implement a CalendarView with colour-coded date markers (Red for Booked, Green for Free, Amber for Pending) that synchronises with Firebase Realtime Database so all users see the same live availability.
- **Prevent double booking** – Implement a server-side conflict check in Firebase that rejects any booking request for a date and time slot that is already confirmed, guaranteeing that no two bookings can exist for the same slot.
- **Enable digital booking requests** – Provide a simple booking request form collecting requester name, contact number, event type, date, and time slot, submitted to Firebase and visible to the Panchayat admin for approval.
- **Provide a secure admin approval workflow** – Implement a PIN-secured Admin View that allows Panchayat officials to review pending requests and approve or reject them with a single tap, updating the calendar in real time for all users.
- **Maintain a transparent Maintenance Jar** – Allow residents to view a list of maintenance items needed for the hall, see the current pledge amount and target, and add a pledge entry. Display a progress bar for each item showing funds raised versus the target amount.
- **Display a live Event Board** – Show today's confirmed bookings on a scrollable Event Board on the home screen so residents can immediately see what is happening at the hall on any given day.

- **Ensure data integrity across offline periods** – Cache the last Firebase calendar and Event Board data in Room Database so the app provides accurate information even when the device temporarily loses internet connectivity.
- **Protect admin functions** – Implement PIN-based authentication for the Admin View so that only authorised Panchayat officials can approve bookings, reject requests, and manage maintenance items.

6. Features

6.1 Core Features

Feature	Description
PIN-Based Admin Login	4-digit PIN authentication for the Admin View shown when the admin icon is tapped. First-time setup creates the PIN; subsequent access requires the saved PIN. Supports change PIN via Admin Settings. The Community View is accessible to all users without login.
Event Calendar	A full-month CalendarView with colour-coded date markers: Green (Free), Red (Booked/Confirmed), Amber (Pending approval). Tapping a date shows the booking details or a 'Request This Slot' button. Synced in real time with Firebase Realtime Database.
Booking Request Form	A simple form collecting requester name, contact number, event type (Wedding/Meeting/Sports/Other), event date, start time, and end time. Submitted to Firebase as a pending request. Conflict check prevents submission for already-booked slots.
Admin Booking Dashboard	PIN-protected admin screen listing all pending booking requests sorted by date. Each request card shows requester name, event type, date, time, and Approve/Reject buttons. Approval updates the calendar marker from Amber to Red in real time for all users.
Double-Booking Prevention	Firebase transaction-based conflict detection checks for existing confirmed bookings on the requested date and time slot before saving a new request. Conflicting requests are rejected with an error message at the point of submission.
Maintenance Jar	A scrollable list of maintenance items needed for the hall (e.g., 'New ceiling fan – Rs. 500', 'Replace broken chairs – Rs. 1200'). Each item shows a progress bar for funds pledged versus target. Residents can tap an item to pledge a contribution amount. Items stored in Room Database.
Event Board	A real-time scrollable board on the home screen displaying all confirmed bookings for today, sorted by start time. Each entry shows the event type, requester name, and time slot. Helps residents quickly know what is happening at the hall today.
Hall Profile	Admin can save the hall name, Panchayat name, key holder name and contact number, and hall address. The hall name is displayed in the main header and the key holder contact is visible to all community users.

Push Notifications	Booking requesters receive a push notification when their request is approved or rejected by the admin. Implemented using Firebase Cloud Messaging (FCM) device tokens stored against each booking request.
Settings Screen	Admin-only settings for changing the admin PIN, managing hall profile, and toggling notification preferences.
Logout	Admin can securely log out of the Admin View from the 3-dot menu with a confirmation dialog. Session is cleared and the Community View is shown on next launch.

6.2 Future Scope Features

- Online Payment Integration – Allow residents to pay a nominal hall booking fee directly through UPI within the app, with payment confirmation linked to the booking.
- Multi-Hall Support – Extend the app to manage multiple community halls within a single GramPanchayat jurisdiction.
- Multi-Language Support – Extend the interface to support Kannada, Hindi, Tamil, and Telugu in addition to English.
- Booking History Export – Allow the admin to export the full monthly booking log as a PDF report for Panchayat records.
- Recurring Booking Support – Allow approved bookings for recurring weekly events (e.g., yoga class every Saturday) without requiring a new request each week.
- Biometric Authentication – Replace PIN login for the Admin View with fingerprint authentication on supported devices.

7. Functional Requirements

The following functional requirements define the core behaviours the system must support:

FR-1: Authentication

- The system shall display the Community View (calendar and Event Board) to all users without requiring a login.
- Tapping the Admin icon shall display a PIN entry screen.
- First-time admin setup shall prompt the creation of a 4-digit PIN stored in SharedPreferences.
- Subsequent admin access shall require the saved PIN. An incorrect PIN shall display an error message and clear the input field.

FR-2: Event Calendar

- The system shall display a full-month CalendarView with colour-coded date markers reflecting the booking status of each date.
- The calendar shall synchronise booking status in real time from Firebase Realtime Database whenever the device has internet access.
- Tapping a Free date shall present a 'Request This Slot' button. Tapping a Booked date shall display the event type and time slot. Tapping a Pending date shall display 'Awaiting Panchayat Approval'.
- The calendar shall cache the last received Firebase data locally so that dates remain marked correctly when the device is offline.

FR-3: Booking Request

- The system shall provide a booking request form with fields: requester name, contact number, event type, event date, start time, and end time.
- The system shall validate that all fields are completed and that the event date is not in the past before submission.
- Before saving, the system shall perform a Firebase conflict check to verify that no confirmed booking exists for the same date and overlapping time slot.
- On successful submission, the request shall be saved to Firebase with status 'Pending' and the calendar date marker shall change to Amber.

FR-4: Admin Booking Dashboard

- The Admin Dashboard shall display all booking requests with status 'Pending', sorted by event date ascending.
- Each request card shall show requester name, contact number, event type, date, and time slot, with Approve and Reject action buttons.
- On Approve, the system shall update the Firebase booking status to 'Confirmed', change the calendar marker to Red, and send a push notification to the requester.
- On Reject, the system shall update the status to 'Rejected', remove the Amber marker from the calendar, and send a rejection notification to the requester.

FR-5: Double-Booking Prevention

- The system shall use a Firebase transaction to atomically check for and prevent conflicting booking confirmations.
- A booking request shall be rejected at the submission stage if a confirmed booking already exists for any overlapping time on the same date.
- The system shall display a clear error message to the user when a slot conflict is detected.

FR-6: Maintenance Jar

- The system shall display a scrollable list of maintenance items stored in Room Database.
- Each maintenance item shall show the item name, target amount (■), total pledged amount, and a progress bar.
- Admin shall be able to add, edit, and delete maintenance items from the Admin View.
- Any community user shall be able to tap a maintenance item and pledge a contribution amount. Pledges shall update the total pledged amount and progress bar immediately.

FR-7: Event Board

- The home screen shall display a scrollable Event Board showing all confirmed bookings for the current date, sorted by start time.
- Each Event Board entry shall display the event type, requester name, and time slot.
- The Event Board shall update in real time when Firebase data changes.

FR-8: Hall Profile and Settings

- Admin shall be able to save the hall name, Panchayat name, key holder name, key holder contact, and hall address.
- The hall name shall be displayed in the main app header visible to all users.
- The key holder name and contact number shall be visible to all community users on the home screen info card.
- Admin shall be able to change their PIN from the Settings screen.

8. Non-Functional Requirements

Category	Requirement	Acceptance Criterion
Performance	Load Time	Community View calendar must fully render within 1000 ms of app launch on a mid-range Android device with an active internet connection.
Performance	Booking Submission	A booking request must be submitted to Firebase and the calendar marker updated within 2000 ms of tapping Submit on a standard mobile data connection.
Usability	UI Simplicity	Any community user must be able to submit a booking request within 5 taps from the app home screen.

Usability	Legibility	All primary text must use a minimum font size of 13sp. Calendar date numbers must use a minimum of 16sp.
Reliability	Data Integrity	Double-booking prevention must succeed in 100% of conflict scenarios tested, including simultaneous submissions from two devices.
Reliability	Crash Rate	Application crash rate must not exceed 1% during standard booking, approval, and maintenance flows.
Accessibility	Touch Targets	All interactive elements must have a minimum touch target size of 48x48dp per Material Design guidelines.
Compatibility	Android Versions	Application must support Android 5.0 (API 21) through Android 14 (API 34).
Security	Admin Access	All admin functions (approve, reject, add maintenance items) must be inaccessible without the correct 4-digit PIN. PIN is stored in SharedPreferences.
Offline Support	Degraded Mode	The calendar and Event Board must display the last cached data when the device is offline. Maintenance Jar features must function fully offline using Room Database.

9. System Architecture

Grama-Angana follows a two-tier client-server architecture for booking management and a single-tier client-only architecture for maintenance tracking. Booking data is managed through Firebase Realtime Database, providing real-time synchronisation across all user devices. Maintenance item data is stored locally using Room Database for offline resilience. All UI processing and business logic occur on the client device.

9.1 Technology Stack

Layer	Technology	Purpose
Language	Kotlin	Primary development language; concise, null-safe, fully interoperable with Android SDK.
UI Framework	XML Layouts + AppCompatActivity + Material	View-based UI with custom CalendarView, RecyclerView, CardView, ProgressBar, and Material components for broad device compatibility.
Real-time Database	Firebase Realtime Database	Stores and synchronises booking requests and confirmed bookings in real time across all user devices. Supports offline caching via Firebase's built-in persistence.

Push Notifications	Firebase Cloud Messaging (FCM)	Delivers booking approval and rejection notifications to requesters using FCM device tokens stored with each booking entry.
Local Database	Room Database (SQLite)	Stores maintenance items, pledge records, and hall profile data permanently on-device for offline access.
Session Storage	SharedPreferences	Stores admin PIN, login state, and hall profile data as lightweight key-value pairs.
Build System	Gradle (Groovy DSL)	Dependency management, build configuration, and APK generation including kotlin-kapt for Room annotation processing.
Min. SDK	API Level 21	Targets approximately 98% of active Android devices globally, including low-cost smartphones prevalent in rural India.

9.2 Architectural Overview

The application follows a simplified MVC (Model-View-Controller) pattern with a Firebase data layer for real-time booking management:

- **Model Layer** – The data package contains Room @Entity classes (MaintenanceItem, PledgeEntry, HallProfile), @Dao interfaces (MaintenanceDao, PledgeDao), and the @Database singleton (AppDatabase). The Firebase package contains the BookingRepository class that wraps all Firebase Realtime Database read and write operations.
- **View Layer** – XML layout files define the UI structure (activity_main.xml, activity_login.xml, activity_booking_form.xml, activity_admin_dashboard.xml, activity_maintenance.xml, activity_hall_profile.xml, activity_settings.xml, list_item_booking.xml, list_item_maintenance.xml). Custom drawables provide CalendarView day markers (green/red/amber) and progress bar styling.
- **Controller Layer** – Activity classes (MainActivity, LoginActivity, BookingFormActivity, AdminDashboardActivity, MaintenanceActivity, HallProfileActivity, SettingsActivity) orchestrate user interaction, invoke Firebase and Room Database operations on background threads, and update the UI via runOnUiThread or Firebase ValueEventListeners.
- **Firestore Listener** – BookingRepository attaches a ValueEventListener to the Firestore 'bookings' node. Any change to booking status triggers a callback that updates the CalendarView markers and Event Board in real time on the main thread.

9.3 Firestore Data Structure

The Firestore Realtime Database uses the following JSON node structure:

Node Path	Field	Type	Description
/bookings/{bookingId}	requesterName	String	Full name of the community member requesting the slot.

	contactNumber	String	Phone number for admin contact and FCM notification delivery.
	eventType	String	Category: Wedding, Meeting, Sports, Health Camp, Other.
	eventDate	String	ISO 8601 formatted date (yyyy-MM-dd) of the requested booking.
	startTime	String	Booking start time in HH:mm 24-hour format.
	endTime	String	Booking end time in HH:mm 24-hour format.
	status	String	Booking lifecycle state: Pending, Confirmed, or Rejected.
	fcmToken	String	Device FCM token for push notification delivery on status change.
	submittedAt	Long	Unix timestamp of submission for conflict resolution and audit log.

The Room Database contains two tables — `maintenance_items` and `pledge_entries` — storing local maintenance data. `maintenance_items` holds: `id` (PRIMARY KEY AUTO), `itemName` (TEXT NOT NULL), `targetAmountRs` (INTEGER NOT NULL), `totalPledgedRs` (INTEGER DEFAULT 0), `addedAt` (TEXT NOT NULL). `pledge_entries` holds: `id`, `maintenanceItemId` (foreign key), `pledgerName` (TEXT), `pledgeAmountRs` (INTEGER NOT NULL), `pledgedAt` (TEXT NOT NULL).

10. User Flow

The application supports two distinct user journeys: the Community View for residents (no login required) and the Admin View for Panchayat officials (PIN-secured). Both journeys are described below:

#	Screen	User Action & System Response
1	Home Screen	App launches directly to the Community View showing the event calendar, Event Board for today, and the key holder contact card. No login required.
2	Event Calendar	Resident views the month calendar with colour-coded dates. Green dates are available, Red dates are confirmed bookings, Amber dates have pending requests. Firebase listener updates markers in real time.
3	View Booking Detail	Resident taps a Booked (Red) date to view the event type and time slot. Taps a Pending (Amber) date to see 'Awaiting Panchayat Approval'. Taps a Free (Green) date to see the 'Request This Slot' button.

4	Booking Request Form	Resident fills in name, contact, event type, date, start and end time. Taps Submit. System checks Firebase for conflicts. On success, request is saved and the calendar date changes to Amber.
5	Event Board	Resident views the Event Board on the home screen showing today's confirmed events sorted by start time. Taps an event entry to see full booking details.
6	Maintenance Jar	Resident taps Maintenance Jar from the bottom navigation. Views items needed with progress bars. Taps an item to see pledges and add a pledge contribution.
7	Admin Login	Panchayat official taps the Admin icon. PIN entry screen appears. Enters correct PIN to access the Admin Dashboard.
8	Admin Dashboard	Admin views all pending booking requests. Taps Approve — Firebase status updates to Confirmed, calendar date turns Red, and requester receives a push notification. Taps Reject — calendar clears and requester is notified.
9	Manage Maintenance	Admin taps + to add a new maintenance item with name and target amount. Long-presses an item to edit or delete it with a confirmation dialog.
10	Admin Logout	Admin taps 3-dot menu → Logout. Confirmation dialog shown. On confirm, session is cleared and the Community View is shown.

11. Technical Implementation

11.1 Firebase Realtime Database Integration

BookingRepository.kt encapsulates all Firebase Realtime Database interactions. The repository holds a reference to the 'bookings' node via `FirebaseDatabase.getInstance().getReference('bookings')`. On MainActivity creation, the repository attaches a `ValueEventListener` that parses all child nodes into Booking data objects and posts the list to a mutable LiveData-style callback. MainActivity uses this callback to update the CalendarView markers and Event Board RecyclerView on the main thread via `runOnUiThread`. Firebase's built-in offline persistence (`FirebaseDatabase.getInstance().setPersistenceEnabled(true)`) caches the last received data locally, ensuring the calendar displays correctly even when the device temporarily loses connectivity.

11.2 Double-Booking Prevention

Conflict detection is implemented using Firebase Transactions. When a booking request is submitted, BookingRepository runs a `Firebase runTransaction()` call on the 'bookings' node. The transaction handler retrieves the current data snapshot, iterates all existing confirmed bookings on the same `eventDate`, and checks for time overlap using interval intersection logic: overlap exists if `requestedStart < existingEnd AND requestedEnd > existingStart`. If any overlap is found, the transaction aborts and returns a `TaskResult.abort()`, triggering the conflict error callback to BookingFormActivity. If no conflict exists, the transaction commits the new booking with status 'Pending'. This atomic approach ensures that simultaneous submissions from two devices cannot both succeed for the same slot.

11.3 Custom CalendarView with Status Markers

The CalendarView is implemented as a custom View class (BookingCalendarView.kt) that extends View and draws a grid of month dates using Canvas. Each date cell is drawn with a circular background colour determined by a statusMap: Map keyed by ISO date string (yyyy-MM-dd) and valued by booking status ('Free', 'Pending', 'Confirmed'). Green, Amber, and Red circular drawables are applied to the corresponding date cell backgrounds. The statusMap is updated by MainActivity whenever Firebase data changes, triggering invalidate() to redraw the calendar. Touch events on date cells are detected using the date cell bounding rectangles computed during the draw pass.

11.4 Firebase Cloud Messaging for Notifications

Each device registers an FCM token on app launch using FirebaseMessaging.getInstance().token. When a user submits a booking request, their FCM token is saved alongside the booking entry in Firebase. When the admin approves or rejects a booking, the AdminDashboardActivity retrieves the FCM token from the booking record and dispatches a targeted FCM data message via the Firebase Admin SDK invoked through a lightweight Firebase Cloud Function. The notification payload includes the booking date, event type, and approval status. On the device, the notification is displayed using a NotificationCompat.Builder with a custom notification channel registered on Android 8.0 and above.

11.5 Room Database for Maintenance Items

Maintenance items and pledge records are stored using the Room persistence library. MaintenanceDao exposes getAll() (ordered by addedAt DESC), insert(), update(), and delete() for maintenance items. PledgeDao exposes insert() and getByItemId(itemId) to retrieve all pledges for a specific item. The totalPledgedRs field on each MaintenanceItem is updated by summing all PledgeEntry amounts via a background Executor query whenever a new pledge is added. MaintenanceActivity uses a RecyclerView backed by a custom MaintenanceAdapter that computes and renders the progress bar percentage (totalPledgedRs / targetAmountRs * 100) for each item in getView().

11.6 Background Threading

Room Database operations are executed on a single-threaded Executors.newSingleThreadExecutor() instance to comply with Room's main-thread restriction. Firebase ValueEventListener deliver their callbacks on the main thread automatically. All UI updates following background database operations are posted back to the main thread via runOnUiThread{}. This pattern avoids the need for LiveData or coroutines while maintaining thread safety across both the Firebase and Room data layers.

11.7 Security – Admin PIN Authentication

The 4-digit admin PIN is stored in SharedPreferences under a named preferences file (GramaAnganaPrefs). The admin session state (isAdminLoggedIn boolean) is set to true on successful PIN entry and cleared to false on logout. LoginActivity checks this flag on creation and routes directly to AdminDashboardActivity if a session is active, avoiding repeated PIN entry during a single administrative session. The Community View is always accessible without authentication. PIN hashing using SHA-256 is planned as a security enhancement in a future version.

12. Project File Structure

File / Directory	Purpose
app/build.gradle	App-level Gradle config: plugins, SDK versions, Room, Firebase BOM, FCM, and kapt dependencies.
google-services.json	Firebase project configuration file enabling SDK authentication and database endpoint routing.
data/Maintenanceltem.kt	Room @Entity data class representing a single maintenance item with target and pledged amounts.
data/PledgeEntry.kt	Room @Entity data class representing a single community pledge for a maintenance item.
data/MaintenanceDao.kt	Room @Dao interface with getAll, insert, update, and delete queries for maintenance items.
data/PledgeDao.kt	Room @Dao interface with insert and getByItemId queries for pledges.
data/AppDatabase.kt	Room @Database singleton with version management and migration definitions.
firebase/Booking.kt	Data class representing a single booking entry as stored in Firebase Realtime Database.
firebase/BookingRepository.kt	Encapsulates all Firebase read, write, and transaction operations for booking management and conflict detection.
LoginActivity.kt	Admin PIN creation and authentication screen. Routes to AdminDashboardActivity on success.
MainActivity.kt	Community View home screen: CalendarView, Event Board RecyclerView, key holder info card, bottom navigation, Firebase listener.
BookingFormActivity.kt	Booking request form: name, contact, event type, date picker, time pickers, conflict check, and Firebase submission.
AdminDashboardActivity.kt	Admin-only pending requests list: Approve/Reject actions, FCM notification dispatch, Firebase status updates.
MaintenanceActivity.kt	Maintenance Jar screen: item list with progress bars, add/edit/delete for admin, pledge submission for all users.

HallProfileActivity.kt	Admin form for saving hall name, Panchayat name, key holder details, and hall address to SharedPreferences.
SettingsActivity.kt	Admin-only settings for PIN change and notification preferences.
BookingCalendarView.kt	Custom View implementing the month calendar grid with colour-coded date cell backgrounds drawn via Canvas.
MaintenanceAdapter.kt	Custom RecyclerView.Adapter inflating list_item_maintenance.xml and computing progress bar percentage for each item.
BookingAdapter.kt	Custom RecyclerView.Adapter inflating list_item_booking.xml for the Event Board and Admin Dashboard booking lists.

File / Directory	Purpose
MyFirebaseMessagingService.kt	FirebaseMessagingService subclass handling FCM token refresh and incoming message display via NotificationCompat.
res/layout/activity_main.xml	Community View layout: header, BookingCalendarView, Event Board RecyclerView, key holder card, bottom navigation.
res/layout/activity_admin_dashboard.xml	Admin Dashboard layout: pending bookings RecyclerView with Approve/Reject action buttons.
res/drawable/*.xml	Shape drawables for booking status circles (green/red/amber), card backgrounds, progress bar track, and button styles.
res/menu/main_menu.xml	3-dot overflow menu items: Admin Login, Hall Profile, Logout.
AndroidManifest.xml	App manifest: permissions, activity declarations, FCM service registration, and launcher activity configuration.

13. Permissions & Privacy

Permission	Required For	Notes
INTERNET	Firebase Realtime Database sync and FCM push notifications	Normal permission; granted automatically. Required for booking synchronisation and notification delivery.
POST_NOTIFICATIONS	Booking approval/rejection push notifications	Required on Android 13+. User prompted on first notification trigger.
RECEIVE_BOOT_COMPLETED	Restore FCM listener after device reboot	Ensures booking notifications are received after a device restart.

No sensitive personal data is transmitted beyond what is required for the booking and notification functions. Booking requests stored in Firebase contain only the requester's name, phone number, event type, date, and time — no financial, health, or identity data. Maintenance item pledges are stored locally in Room Database and are not transmitted to any external server. The admin PIN is stored locally in SharedPreferences and is never transmitted. The application does not collect analytics or usage statistics.

14. Impact Goals

14.1 Asset Utilisation – Maximising Public Infrastructure Value

By making hall availability instantly visible to every community member, Grama-Angana directly reduces the periods of unnecessary vacancy that cause public buildings to fall into disuse and decay. A hall that is easy to book is a hall that is regularly used — for sports training, health camps, cultural events, and community meetings — generating social and economic returns on the government's infrastructure investment.

14.2 Social Capital – Bringing Communities Together

The Event Board and Maintenance Jar features transform the app from a booking tool into a community notice board and collective action platform. When residents can see what is happening at their hall today and contribute to fixing a broken fan, they develop a shared stake in their community space. This sense of collective ownership strengthens social bonds and encourages voluntary civic participation in ways that top-down government programmes rarely achieve.

14.3 Transparency – Eliminating Favouritism in Public Bookings

The real-time, publicly visible booking calendar is the application's most powerful anti-corruption feature. When every community member can see which dates are booked, by whom, and when the booking was confirmed, the informal back-channel process of allocating hall access based on personal connections becomes impossible to sustain. Transparency in a simple booking calendar is, in effect, a governance reform tool.

14.4 Digital Inclusion in Local Governance

The application demonstrates that effective e-governance tools for Gram Panchayats do not require expensive servers, dedicated IT staff, or high digital literacy. By using Firebase's free tier for small communities and targeting API 21 and above, Grama-Angana brings a professional booking and maintenance management system to village-level governance at near-zero cost.

14.5 Scalability of Impact

The modular architecture of Grama-Angana — with a separate Firebase booking layer, Room maintenance layer, and custom CalendarView — supports rapid expansion. Future versions can add multi-hall support, UPI booking fee payment, multi-language interfaces, and Panchayat-level reporting dashboards without requiring a fundamental redesign of the existing codebase.

15. Success Criteria

The project will be considered successfully completed when the following criteria are met:

#	Success Criterion	Measurement	Priority
1	App loads and displays the Community View calendar within 1000 ms on a reference device (Redmi 9A or equivalent) with mobile data.	Android measurement Profiler	Critical

2	The calendar correctly prevents double booking — two simultaneous submissions for the same slot result in only one confirmation.	Concurrent two-device submission test	Critical
3	A community user can submit a complete booking request within 5 taps from the home screen.	Manual UI walkthrough	Critical
4	Admin approval of a booking updates the calendar marker from Amber to Red on both the admin device and a second community device within 3 seconds.	Two-device real-time sync test	Critical
5	PIN login correctly blocks admin access with wrong PIN and grants access with correct PIN.	Manual security testing	Critical
6	Booking requester receives a push notification within 10 seconds of admin approving or rejecting their request.	FCM delivery log and manual device test	High
7	The Maintenance Jar progress bar updates correctly after each pledge entry is saved.	Manual pledge entry and progress bar validation	High
8	The Event Board shows only today's confirmed bookings, sorted by start time, and updates immediately when a new booking is confirmed.	Manual validation with test bookings	High
9	App does not crash during a complete walkthrough of all screens and features including admin and community flows.	Manual regression testing	High
10	The calendar and Maintenance Jar display correctly in Airplane Mode using last cached Firebase data and local Room data.	Airplane Mode test on device	Medium

Conclusion

Grama-Angana represents a focused and socially meaningful application of Android development in the domain of community infrastructure management and rural e-governance. By addressing a clear and well-defined problem — the lack of a transparent, real-time, and community-accessible hall booking system for India's village Anganas and Samudaya Bhavanas — the project delivers measurable civic value alongside technical merit.

The application's architecture balances the real-time synchronisation power of Firebase Realtime Database for booking management with the offline resilience of Room Database for maintenance tracking.

Every design decision, from the custom colour-coded CalendarView for intuitive availability display to the Firebase transaction-based double-booking prevention and the Maintenance Jar crowdfunding feature, is

motivated by a single question: will this make the village hall more accessible, more fairly managed, and better maintained for every member of the community?

The double-booking prevention mechanism and the publicly visible booking calendar together constitute the application's defining civic innovation. By replacing an opaque, paper-based, relationship-driven booking process with a transparent, digital, first-come-first-served system visible to every community member, Grama-Angana achieves in a simple Android app what governance reformers have long sought to accomplish through policy intervention alone.

The project also establishes a strong technical foundation for future expansion. The separation of the Firebase booking layer, Room maintenance layer, custom CalendarView, and controller activities supports the addition of multi-hall support, UPI payment integration, Panchayat reporting dashboards, and multi-language interfaces without requiring a fundamental redesign. The Room migration framework and Firebase's versioned data structure ensure that future changes can be deployed without data loss.

In conclusion, Grama-Angana is not simply a technical deliverable — it is a small but meaningful step toward transparent local governance, stronger community bonds, and the responsible stewardship of public infrastructure for the millions of Indian villages whose community halls stand empty not for lack of need, but for lack of a simple, fair, and visible way to use them.